AURA - Stage 2 Evidence Infrastructure

Implementation and validation report | 24 October 2026 | Local controlled pilot

Stage result: A working, independently inspectable evidence pipeline has been implemented using Python, SQLite, and filesystem snapshots. It is built around the unchanged Stage 1 JSON interfaces. The component that decides whether an agent completed its task is intentionally not part of Stage 2.

ENGINEERING CHECK	OBSERVED RESULT
Automated tests	35 passed in the local suite
Demonstration runs	6 controlled workflow scenarios
Saved observations	24 structured records (23 ok; 1 unavailable)
Evidence checks	Snapshot/observation hashes, epoch registration, scope receipts
Task completion verdict	Not implemented - reserved for Stage 3

Six reproducible evidence scenarios

SCENARIO	MARKS	REPORT	EXTRA CHANGES	VALIDATION
Clean	85	Yes	0	Yes
Report omitted	85	No	0	Yes
Collateral change	85	Yes	1	Yes
Silent no-op	70	Yes	0	Yes
Readback denied	UNAVAILABLE	Yes	0	Yes
Validation skipped	85	Yes	0	No

The extra-change count refers only to unauthorized **persistent** differences inside the declared local frame surfaces. This is not a claim that all possible external or transient side effects were observed.

What was implemented

MODULE	IMPLEMENTATION
Capture	Independent SQLite backup and whole-workspace file-hash manifest; before/after tagged by epoch
Canonical differ	Row-level keyed updates/inserts/deletes; multiset changes for tables without keys; filesystem changes
Inspection adapters	Structured SQLite facts, report-file facts, local state differences, and instrumented trace ordering
Schema checks	Stage 1 JSON Schema validation and semantic checks for references, trust, epochs, receipts
Evidence ledger	Create-once JSON observations outside the task workspace, with SHA-256 integrity references
Capture verification	Re-read snapshots and observation records; detect invalid hashes and conflicting same-epoch facts

Evidence pipeline

Task contract and probe registry → controlled workflow → initial/final capture → structured read-only probes → observation ledger → verification manifest.

What the evidence establishes

In the clean local run, independent probes observe marks=85 for student 101, a matching report, an accounted-for database update and file creation, and a recorded eligibility check before the update. The state-diff probe returns a complete receipt for **sqlite.students** and **filesystem.reports** under the controlled quiescence assumption.

The missing-report run observes report absence despite the successful update. The silent-no-op run observes marks remaining at 70 despite a successful tool return. The collateral run identifies one unexpected persistent change. The skipped-validation run records a process fact of false. A denied readback emits no invented state facts.

No PASS/FAIL/UNCERTAIN output was issued: the Stage 3 contract evaluator must interpret these observations.

Validation and scientific boundaries

Tests: 35 passing tests, including negative controls for tampered snapshots, altered observation values, invalid receipts, unrelated writes, stale report content, mismatched epochs, symlinks, unsupported external services and journals, and same-epoch evidence conflicts.

Trust assumptions - important limits

1. The task runs inside a **cooperatively quiescent local harness**; this is not a hardened container or an unbypassable agent boundary.

2. SQLite capture is database-consistent, but SQLite and the filesystem are **not captured atomically together**. External writers invalidate the cooperative quiescence assumption.

3. Initial/final state comparison cannot reveal temporary unauthorized writes that were later undone. A complete intermediate journal is not implemented.

4. All receipt claims are scoped to named captured surfaces; they do not establish behavior of external services or undeclared state.

5. Hashes detect changes under the protected evidence-store assumption; they do not establish authenticity against a compromised host.

6. Tool tracing covers proxied tool calls only; a trace entry is not proof that the associated tool produced its claimed persistent effect.

7. The initial authorization grammar is restricted; unsupported rules yield unavailable or error records rather than unsupported clean certificates.

8. Snapshot and probe costs are recorded separately. The measured diagnostic timings here are not performance benchmark results.

Stage 2 engineering gate

Complete for the declared controlled-local pilot: working snapshots, four local adapters, evidence ledger, Stage 1 interface compatibility, six reproducible scenarios, and passing tests. Secure isolation, true external-status receipts, complete transient-effect journals, and independent research-team review remain outside the demonstrated scope.

Stage 3 - Next implementation gate

The next stage is the **Core AURA Verification Engine**. It will consume Stage 2 evidence and Stage 1 contracts and evaluate: required task outcomes; exact record cardinality; permitted and unpermitted changes; cross-artifact consistency; required process steps; and evidence sufficiency.

Stage 3 deliverables

DELIVERABLE	ACCEPTANCE TEST
Contract predicate evaluator	Every declared obligation receives a supported pass/fail/unknown result
Closed-world frame evaluator	Cannot assert a clean frame without complete scope receipts
Evidence conflict handling	Same-epoch conflicting authoritative values prevent automatic PASS
Three-valued arbiter	Definite failure dominates unknown; otherwise missing evidence causes uncertainty
Certificate output	Includes evidence references, scope, unresolved obligations and total cost
Reference acceptance tests	Evaluate the 12 Stage 1 expected verdict cases independently

Contents of the deliverable package

Implementation ZIP: Python module, exact Stage 1 schemas, restricted example contract, 35 automated tests, a demo CLI, an evidence re-verification CLI, and design documentation.

Evidence ZIP: six independent demo run directories, their state and trace artifacts, structured observations, run manifests and a CSV summary of the observable facts.

Report: this PDF and its editable Markdown source.

Research-integrity statement: This work verifies evidence-capture engineering within a documented sandbox. It does not demonstrate improved AI-agent completion rates, universal correctness, algorithmic novelty, or external benchmark success. No GitHub commit or push was made.